LAB # 3
To design the finite state machines using VHDL and reproduce the results on FPGA board

Objectives

The objective of this lab is to design and test Finite State Machines (FSMs) using VHDL on an FPGA board. By the end of this session, you will:

- **Transition** from thinking in logic gates to high-level functional states.
- **Implement** FSMs using the professional **3-Process Architecture**.
- **Compare** the timing and behavior of **Mealy** and **Moore** machines.

Pre-Lab

1. Introduction: Thinking in States vs. Gates

As a designer, you are moving beyond simple combinational logic and basic sequential counters. The shift to **Finite State Machines (FSMs)** represents a transition from "What gate do I need?" to "What functional state is the system in?".
While a counter follows a fixed, repetitive sequence, an FSM acts as the **Command and Control** unit for complex digital systems. It evaluates current conditions (Inputs) and its history (Present State) to decide the next action. In high-level design, we do not manually map flip-flops; we describe the **behavioral transitions** and let the synthesis tool optimize the underlying gates. There are two ways to design a finite state machine (FSM); Mealy FSM and Moore FSM.

Introduction, specification, design and testing of Mealy FSM

There are two basic types of sequential circuits: Mealy and Moore. Because these circuits transit among a finitenumber of internal states, they are referred to as finite state machines (FSMs). In a Mealy circuit, the outputs depend on both the present inputs and state. In a Moore circuit, the outputs depend only on the present state. The most common way of schematically representing a Mealy sequential circuit is shown in Fig.3.1

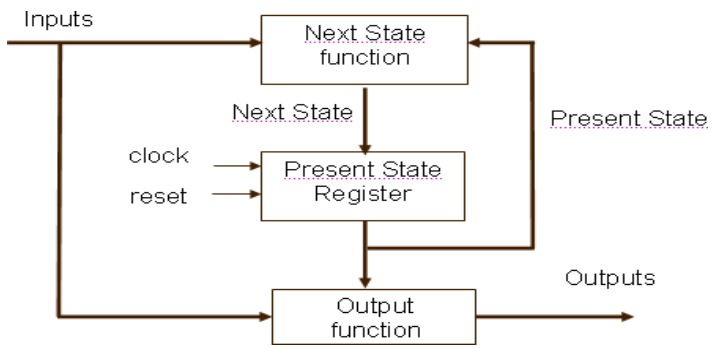

FIGURE 3.1 GENERALIZED MEALY SEQUENTIAL CIRCUIT

The state register normally consists of D flip-flops (DFFs). However, other types of flip-flops can be utilized, such as JKFFs. The normal sequence of events is: (1) inputs X change to a new value, (2) after a clock period delay, outputs Z and next state NS become stable at the output of the combinational circuit, the next state signalsNS are stored in the state register; that is, next state NS replace present state PS at the output of the state register,which feeds back into the combinational circuit. At this time, a new cycle is ready to start. These operational cycles are synchronized with the clock signal CLK.

It is worth mentioning that some authors further classify sequential circuits into two categories. The first category, referred to as “regular sequential circuits”, includes circuits like (shift) registers, FIFOs, and binary counters and variants. The second category, referred to as “finite state machines” (FSMs), include circuits thattypically do not exhibit a simple, repetitive pattern.

2. The 3-Process Implementation Architecture

To ensure code maintainability and timing reliability, this lab adopts the **3-Process Style**. This methodology separates the FSM into three distinct logical blocks:

I. The State Register (Synchronous)

This process is the "heartbeat" of the FSM. It uses a clock edge to move the machine from the current_state to the next_state.

- **Purpose:** To store the machine's progress.
- **Template:**

```
PROCESS(clk, reset)
BEGIN
    IF reset = '1' THEN
        current_state <= IDLE; -- Always define a known starting point
    ELSIF rising_edge(clk) THEN
        current_state <= next_state; [cite: 211]
    END IF;
END PROCESS;
```

II. **Next State Logic (NSL - Combinational)**

This block determines where the machine goes next based on the current_state and the inputs.

- **Critical Rule:** The sensitivity list must include both the current_state and all relevant inputs to avoid simulation-synthesis mismatches.

III. **Output Function Logic (OFL - Combinational)**

This block drives the system outputs based on the state (and inputs, if Mealy).

Warning: Unintentional Latches In combinational processes (NSL and OFL), every possible path **must** assign a value. If a state or input combination is missing an assignment, the FPGA will "infer a latch" to hold the previous value. Always use WHEN OTHERS => and ensure every IF has an ELSE.

3. Moore vs. Mealy: Architectural Trade-offs

The lab requires implementing both styles to understand their timing implications.

Feature	Moore Machine +1	Mealy Machine
Output Dependency	Only on the **Present State**.	On **Present State** AND **Inputs**.
Timing	Outputs change only on clock edges.	Outputs can change "instantly" with inputs.
Stability	High; immune to input glitches.	Risk of asynchronous glitches (hazards).
Complexity	Often requires more states.	Can often be done with fewer states.

4. Enumerated Data Types

Hard-coding state values (e.g., 00, 01) is a legacy practice that leads to "magic number" errors. Instead, use **Enumerated Types** to define states by their logical names:

```
TYPE state_type IS (IDLE, START, PROCESSING, DONE);
```

ModelSim Debugging Tip: When simulating, your waveform will display "IDLE" or "DONE" instead of binary bits, making debugging significantly faster. If names do not appear, right-click the signal in the **Wave window** and select **Radix > Symbolic**.

In-Lab Tasks:

Task 1: MEALY machine design – BCD to Excess-3 code converter

In this example, we'll design a serial converter that converts a binary coded decimal (BCD) digit to an excess-3-coded decimal digit. Excess-3 binary-coded decimal (XS-3) code, also called biased representation or Excess-N, is a complementary BCD code and numeral system. It was used on some older

computers with a pre-specifiednumber N as a biasing value. It is a way to represent values with a balanced number of positive and negative numbers. In our example, the XS-3 code is formed by adding 0011 to the BCD digit. The table and state graphin Fig.3.2 describe the functionality of our design.

X Input (BCD)				Z Output (excess-3)			
t_3	t_2	t_1	t_0	t_3	t_2	t_1	t_0
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

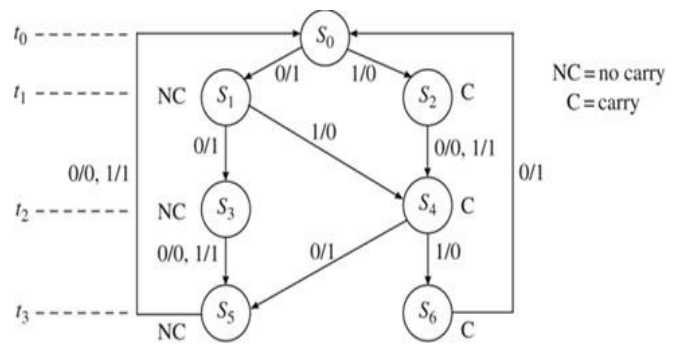

FIGURE 3.2 TRUTH TABLE AND FINITE STATE MACHINE OF BCD TO EXCESS-3 CODE CONVERTER

There are several ways to model this sequential machine. One popular/common approach is to use **two processes** to represent the two parts of the circuit: the combinational part and the state register. For clarity andflexibility, we use VHDL's *enumerated data type* to represent the FSM's states. The following VHDL code describes the converter.

Task 2: Introduction, specification, design and testing of Moore FSM

In a Moore circuit, the outputs depend only on the present state. The most common way of schematically representing a Mealy sequential circuit is shown in Fig.3.3

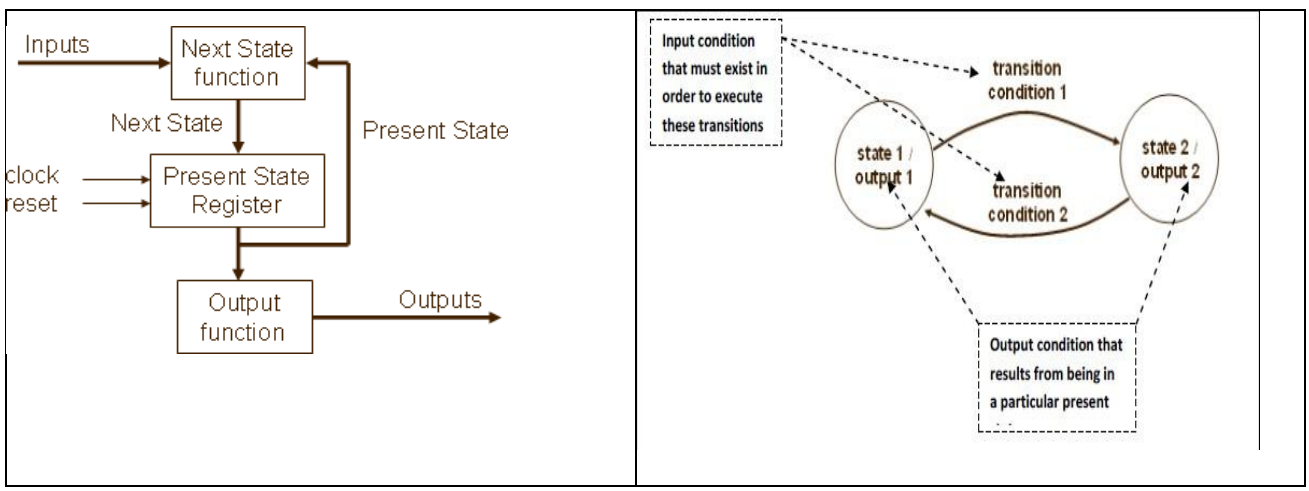

FIGURE 3.3 GENERALIZED MOORE SEQUENTIAL CIRCUIT

In this task, you will be required to understand and complete the behavioral VHDL code for the Moore FSM. The state diagram for the FSM is given below. Since the synthesizer will automatically take care of the state encoding, therefore, the states only need to be labelled with their logical names.

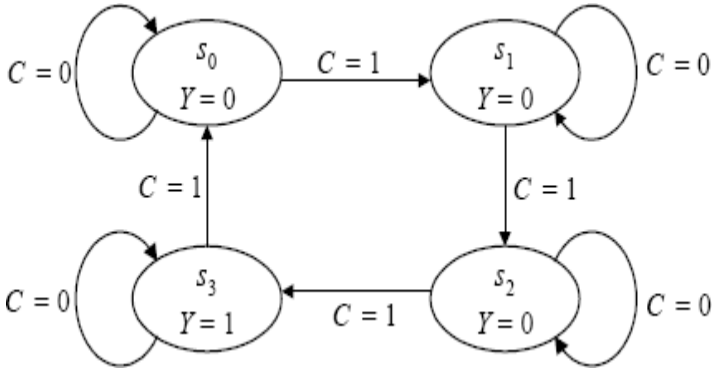

FIGURE 3.4 MOORE FINITE STATE MACHINE-AN EXAMPLE

Rubric for lab assessment

The student performance for the assigned task during the lab session was:			
Excellent	The student completed all the tasks and showed the results withoutany help of the instructor.	4	
Good	The student completed all the tasks and showed the results with minimal help of the instructor.	3	
Average	The student partially completed the task and showed results.	2	
Worst	The student did not complete the task.	1	

Instructor Signature:_____ Date: _____